

Constrained Pseudorandom Functions, Revisited

Julian Mouthon Mario Razafinony

Sorbonne Université
Master 1 – Cryptologie, Calcul Haute-Performance et Algorithmes

25 mai 2026

Qu'est-ce qu'une PRF ?

PRF = Pseudorandom Function (fonction pseudo-aléatoire)

C'est une fonction déterministe

$$F_k : \{0, 1\}^n \rightarrow \{0, 1\}^m$$

paramétrée par une clé secrète k .

Sans connaître k , $F_k(x)$ est indistinguable d'une fonction aléatoire.

- Les PRF sont utilisées en cryptographie moderne :
 - chiffrement,
 - authentification,
 - dérivation de clés.
- Problème : comment déléguer seulement une partie des capacités d'évaluation ?
- Solution : les **Constrained Pseudorandom Functions (CPRF)**.

Une CPRF est une PRF dont l'évaluation peut être restreinte.

Exemple de contrainte :

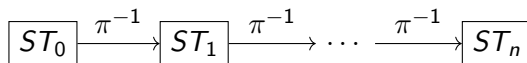
$$C_n(x) = [x < n]$$

- Clé maître : accès complet.
- Clé contrainte : accès limité à certaines entrées.

Objectif : déléguer des capacités sans révéler la clé secrète.

Construction BMO17 : intuition

La construction repose sur une permutation à trappe itérée.



$$Eval(c) = \pi^{-c}(ST_0)$$

Avec la clé secrète, on peut calculer tous les états.

Permutation publique :

$$\pi(x) = x^e \bmod N$$

Inverse de la permutation :

$$\pi^{-1}(x) = x^d \bmod N$$

- La clé privée permet de remonter la chaîne.
- La clé publique permet uniquement d'avancer.
- RSA fournit une permutation bijective adaptée à la construction.

Pour une contrainte $x < n$, on donne :

$$(PK, ST_n, n) \quad \text{avec } ST_n = \pi^{-n}(ST_0)$$

L'évaluation contrainte devient :

$$EvalC(c) = \pi^{n-c}(ST_n)$$

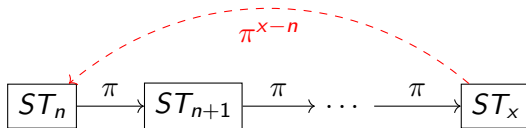
- Possible si $c < n$.
- Impossible d'aller au-delà sans la clé privée.

Attaque CJ25 : idée générale

L'attaquant obtient :

- une clé contrainte (PK, ST_n, n) ,
- des évaluations hors contrainte $Eval(x)$ pour $x > n$.

Puis il exploite la structure publique de la chaîne.



L'attaquant peut vérifier une cohérence impossible dans le monde aléatoire.

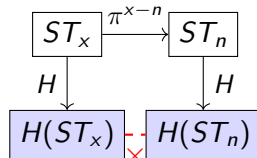
Idée : appliquer une fonction de hachage H sur la sortie de la CPRF.

Évaluation hachée

$$Eval_H(c) = H(\pi^{-c}(ST_0))$$

Évaluation contrainte hachée

$$EvalC_H(c) = H(\pi^{n-c}(ST_n))$$



Génération de la clé contrainte

$$(PK, ST_n, n)$$

BMO17 hachée : pourquoi ça bloque l'attaque?

Rappel de l'attaque CJ25 :

- L'attaquant obtient $Eval(x) = ST_x$ et calcule $\pi^{x-n}(ST_x)$.
- Il compare le résultat à ST_n .

Avec le hachage :

- L'attaquant ne reçoit plus ST_x , mais $H(ST_x)$.
- Il ne peut pas appliquer π^{x-n} sur $H(ST_x)$.
- La comparaison devient **impossible** : l'attaque est bloquée.

Alternative : permutation AES

Idée : remplacer RSA par AES comme permutation à trappe.

AES définit une bijection sur les blocs de 128 bits :

$$f_k(x) = E_k(x), \quad f_k^{-1}(x) = D_k(x)$$

Problème fondamental :

- Pour inverser, il faut **obligatoirement** connaître la clé k .
- Il est impossible de donner une clé contrainte **sans révéler** la clé k .

Alternative : fonction de Rabin

La fonction de Rabin est définie par :

$$f(x) = x^2 \bmod n, \quad n = p \cdot q$$

Inversion (avec la clé secrète (p, q)) : résoudre $x^2 \equiv y \pmod{n}$ donne **4 racines** :

$$\pm r_p \cdot q \cdot (q^{-1} \bmod p) \pm r_q \cdot p \cdot (p^{-1} \bmod q) \pmod{n}$$

Problème fondamental :

- f n'est **pas injective** : une image admet 4 pré-images.
- Même avec la clé privée, on ne sait pas laquelle est la « bonne ».
- Il est donc impossible d'itérer f^{-1} de façon déterministe.

Alternative : F^{WEAK} — construction

Idée : utiliser une application linéaire comme permutation.

Clé maître

$$S \in (\mathbb{Z}/p\mathbb{Z})^{M \times N}$$

matrice choisie aléatoirement.

Évaluation

$$\text{Eval}(S, \vec{x}) = S \cdot \vec{x}$$

Clé contrainte pour $\vec{y}$

L'oracle tire $\vec{d} \neq \vec{0}$ et renvoie :

$$S_{\vec{y}} = S + \vec{d} \cdot \vec{y}^T$$

Évaluation contrainte

$$\text{EvalC}(S_{\vec{y}}, \vec{x}) = S_{\vec{y}} \cdot \vec{x}$$

correcte si et seulement si
 $\langle \vec{x}, \vec{y} \rangle = 0$.

Attaque sur F^{WEAK} — principe

Objectif : retrouver la clé maître S à partir d'une clé contrainte et d'évaluations.

Choix de l'attaquant : demander la clé contrainte pour

$$\vec{y} = (0, \dots, 0, 1)$$

Il reçoit $S_{\vec{y}} = S + \vec{d} \cdot \vec{y}^T$.

Observation clé : pour tout $\vec{x} \perp \vec{y}$,

$$S_{\vec{y}} \cdot \vec{x} = S \cdot \vec{x}$$

Les $N - 1$ premières colonnes de S sont directement accessibles via $S_{\vec{y}}$.

Il reste à retrouver la dernière colonne $S_{\cdot, N}$.

Attaque sur F^{WEAK} — résolution

Pour retrouver $S_{i,N}$: demander $Eval(\vec{x})$ avec $x_N \neq 0$.

Pour chaque ligne i de S , on a :

$$S_{i,1}x_1 + \cdots + S_{i,N-1}x_{N-1} + S_{i,N}x_N = y_i$$

Puisque $S_{i,1}, \dots, S_{i,N-1}$ sont connus, on en déduit :

$$S_{i,N} = x_N^{-1} \left(y_i - \sum_{j=1}^{N-1} S_{i,j}x_j \right)$$

- Si l'oracle utilise une fonction aléatoire : l'attaquant reçoit une valeur aléatoire de $S_{i,N}$
- Si l'oracle utilise F^{WEAK} : l'attaquant reçoit la bonne valeur de $S_{i,N}$
- Après quelques requêtes, l'attaquant retrouve que la bonne valeur $S_{i,N}$ est celle qui revient le plus souvent

Renforcement de F^{WEAK} par hachage

Idée : composer F^{WEAK} avec une fonction de hachage H .

Sans hachage

$$\text{Eval}(\vec{x}) = S \cdot \vec{x}$$

⇒ **attaque linéaire possible.**

Avec hachage

$$\text{Eval}_H(\vec{x}) = H(S \cdot \vec{x})$$

Retrouver S nécessite une pré-image de H .

Théorème (CJ25)

Soit CF une CPRF. Si :

- 1 $|CF.Out(\lambda, x)|$ est super-polynomial en λ ,
- 2 CF est $\text{IND}^*\text{-NC-CPRF}$ sécurisée,
- 3 l'oracle aléatoire est non-programmable,

alors $\text{Hashed}[CF]$ est **$\text{SIM}^*\text{-AC-CPRF}$ sécurisée.**

Condition 1 : $|F^{\text{WEAK}}.Out(\lambda, x)|$ est super-polynomial en λ .

Les coefficients sont dans $\mathbb{Z}/p\mathbb{Z}$ avec $|p| \approx \lambda$ bits.

La matrice S contient $M \times N$ coefficients, donc :

$$|F^{\text{WEAK}}.Out| \approx 2^{\lambda \cdot M \cdot N}$$

✓ **Condition 1 vérifiée.**

Vérification — Condition 3 : oracle non-programmable

Condition 3 : l'oracle aléatoire H est **non-programmable**.

Implémentation d'une fonction de hachage par lazy sampling :

Définition

Un oracle P est non-programmable si :

- on ne peut pas **forcer** $P(z) = y$ pour une valeur y choisie à l'avance,
- la distribution de $P(z)$ pour une entrée non encore évaluée est toujours **uniforme et indépendante**.

- Si z **déjà évalué** : retourner la valeur mémorisée $H(z)$.
- Sinon : tirer $y \leftarrow \{0, 1\}^\lambda$ aléatoirement, mémoriser (z, y) , retourner y .

La valeur de $H(z)$ est fixée **au moment de la première évaluation** et ne peut plus être modifiée.

✓ **Condition 3 vérifiée.**

Condition 2 : F^{WEAK} est IND*-NC-CPRF sécurisée.

Rappel : que signifie IND*-NC ?

La CPRF CF est IND*-NC sécurisée si **aucun attaquant efficace** ne peut :

- 1 demander **toutes** ses évaluations d'un coup (non-adaptativement),
- 2 effectuer ses calculs,
- 3 renvoie sa décision à l'oracle tout en ayant un **avantage significatif**

Pour prouver que F^{WEAK} n'est **pas** IND*-NC sécurisée, il suffit de décrire un tel attaquant.

Vérification — Condition 2 : IND*-NC-CPRF (2/2)

Jeu de sécurité simplifié contre F^{WEAK} :

1. **Clé contrainte.** $\mathcal{A}$ demande $S_{\vec{y}}$ pour $\vec{y} = (0, \dots, 0, 1)$.
 $\Rightarrow \mathcal{A}$ récupère les $N - 1$ premières colonnes de S .
2. **Évaluations (non-adaptatives).** $\mathcal{A}$ envoie **en une seule fois** n vecteurs $\vec{x}^{(1)}, \dots, \vec{x}^{(n)}$ avec $x_N^{(k)} \neq 0$,
et reçoit toutes les réponses $\vec{y}^{(1)}, \dots, \vec{y}^{(n)}$ en retour.
3. **Calcul.** Pour chaque requête k et chaque ligne i , $\mathcal{A}$ calcule :

$$\tilde{S}_{i,N}^{(k)} = (x_N^{(k)})^{-1} \left(y_i^{(k)} - \sum_{j=1}^{N-1} S_{i,j} x_j^{(k)} \right)$$

4. **Décision.** Pour les $\tilde{S}_{i,N}^{(k)}$ **identiques** $\Rightarrow$ CPRF.
Sinon $\Rightarrow$ fonction aléatoire.

✗ **Condition 2 non vérifiée.**

Conclusion sur Hashed[F^{WEAK}]

Condition	Énoncé	Statut
1	Espace de sortie super-polynomial	✓
2	IND*-NC-CPRF sécurisée	✗
3	Oracle non-programmable (lazy sampling)	✓

La condition 2 n'est pas vérifiée : on ne peut pas utiliser ce théorème pour prouver la sécurité de Hashed[F^{WEAK}].

Cela **ne prouve pas** que Hashed[F^{WEAK}] est non sécurisée.

La sécurité de Hashed[F^{WEAK}] reste une **question ouverte**.